

Knowing When to Defer: How Language Models Use and Bypass Sources of Record

Alex Chao
Fide AI
alex@fideai.org

August 2026

Abstract

When someone asks an AI system to quote Scripture, a plausible answer is not enough. The wording, translation, and boundaries of the passage all matter. A model can nevertheless ignore an available Bible source and answer from memory, quietly changing the requested text. This study asks a simple question: what makes a language model use an authoritative source before it answers?

We tested 20 Scripture passages across six model configurations and 4,800 requests. In an experiment specified before data collection, the system either made a Bible lookup tool available or required the model to use it. Users either asked for the passage normally or explicitly asked the model to avoid tools and answer from memory. Because the tool remained technically optional, the action logs showed whether the model called it before answering.

For ordinary requests, models used the source about 95% of the time under both policies; requiring the tool produced no measurable increase. The requirement mattered when the user’s instruction conflicted with it. Source use fell to 30.6% when the tool was merely available, but remained at 84.7% when the system required it—a 54-point difference. This pooled result describes the six tested model configurations, whose effects ranged from 0 to 90.5 points. Tool use was also not a complete guarantee: among 3,664 requests in which models used the source, 89.1% requested the intended passage; when the intended passage was requested, 93.5% reproduced the source text exactly.

Scripture provides a consequential test case because exact wording matters in study, teaching, worship, and pastoral care. The broader lesson is that making an authoritative source available is not the same as ensuring that it governs an AI system’s answer.

1 Introduction

Suppose an AI assistant is asked for Romans 8:28 in a named Bible translation. A Scripture lookup tool is available. Does the model use it? The question sounds like an implementation detail. It decides whether the words the user receives came from the source or from the model’s memory. The model may know the passage well enough to answer immediately. The user may ask it not to “waste time” calling a tool. The system policy may merely offer the source or explicitly require it. In all four cases the same tool is present and the final prose may look convincing. What changes is whether the model treats the source of record as authoritative in action.

For Scripture, that difference has consequences. Christians use quoted passages in teaching, worship, memorization, pastoral communication, and theological argument. A request for a particular translation is a request for particular words, not simply for a semantically similar answer.

A model that recalls a familiar formulation from another edition can preserve the broad meaning while failing the quotation. A model that identifies the right event but calls for a neighboring span of verses can return exact source text and still answer the wrong request. Source use, source selection, and final textual fidelity are related, but they are not the same behavior.

An earlier study in this program established that distinction empirically (Chao, 2026). Across matched quotation requests, connecting an authorized lookup tool raised exact delivery far above generation from model memory, but it did not make the tool self-enforcing. Models could bypass the tool, call it for the wrong reference, or alter the returned text. That study compared end-to-end delivery architectures. The present study holds the architecture fixed and asks a narrower intervention question: *what makes a language model delegate an exact quotation request to the source it has been given?*

Retrieval-augmented generation and tool use are commonly proposed as remedies for the limits of parametric memory (Lewis et al., 2020; Mallen et al., 2023; Schick et al., 2023). A model must still decide whether to invoke a tool, formulate its arguments, and incorporate its result. Two sets of instructions bear on that decision, and they can agree or conflict: the application’s system message and the user’s own request. Work on instruction hierarchies studies exactly this priority ordering. A user may ask a model to answer from memory precisely when the application requires a source check. Answer accuracy alone cannot capture this. A correct answer from memory and a correct answer from the source read identically, but they are different system behaviors.

We study that action with a two-factor experiment specified in advance. In every condition, the model receives the same optional `get_passage` tool backed by a frozen local copy of the Berean Standard Bible. The system message either says that the source is available when useful or requires its use before exact quotation. The user either makes the ordinary quotation request or adds pressure to avoid tools and answer from memory. Twenty passages, two prompt families, six model families, and five repetitions yield 4,800 scheduled observations. We call each evaluated model a *route*: one served endpoint with its upstream provider pinned. Ordered assistant, tool-call, tool-result, and final-answer events make delegation directly observable.

The quantity we set out to estimate is the difference in delegation rate between the source-required and source-available policies under neutral user wording. Secondary contrasts estimate the effect of anti-tool user pressure within each policy and whether the required policy blunts that pressure. Correct-reference delegation and final exact quotation remain separate outcomes. Requiring a tool can change whether a source is consulted without ensuring that the right passage is requested or that the returned words survive the final response.

Scripture is not a stand-in for a generic tooling problem. It is a domain where the obligation is real and checkable. Verse references provide a stable addressing system; English translations make edition identity observable; indirect requests require semantic passage identification; and Christian communities have substantive reasons to care whether quotation marks contain the requested text. At the same time, the experimental structure speaks to any source-governed setting. Statutes, clinical instructions, contracts, standards, and institutional policies all create cases where the model should consult an identified source instead of assuming that fluent recall is enough. The measured effects do not transfer to those domains without replication, but the distinction between source availability and source delegation does.

Contributions. This paper makes four contributions. First, it gives a controlled two-factor contrast of observed delegation under two source-use policies for exact Scripture quotation. Second, it measures conflict between system-level source requirements and user pressure to bypass tools. Third, working from the logged sequence of actions, it separates four things that are easy to

conflate: whether the model called the source, whether it asked for the right passage, whether the passage text appears in the answer, and whether the final quotation is exact. Text appearing in the answer is not treated as proof that the tool’s result produced it. Fourth, it provides a release pattern in which the preregistration, derived scores, analysis, and aggregate evidence can be opened while credentials, raw traces, private execution code, and source text remain protected. The goal is not to rank model vendors. It is to identify which controls make a source of record operative rather than merely present.

In brief. When users asked normally, the two policies were indistinguishable: both produced source use near 95%. When users asked the model to answer from memory instead, the gap opened to 54 points—30.6% under discretionary availability against 84.7% under a system requirement. Even the required policy was bypassed in 15.3% of those requests, and calling the source did not guarantee that the right passage was requested or that the returned words survived intact.

2 From Source Availability to Source Delegation

The tool is optional in every condition

Policy and user pressure change instructions, not technical access.

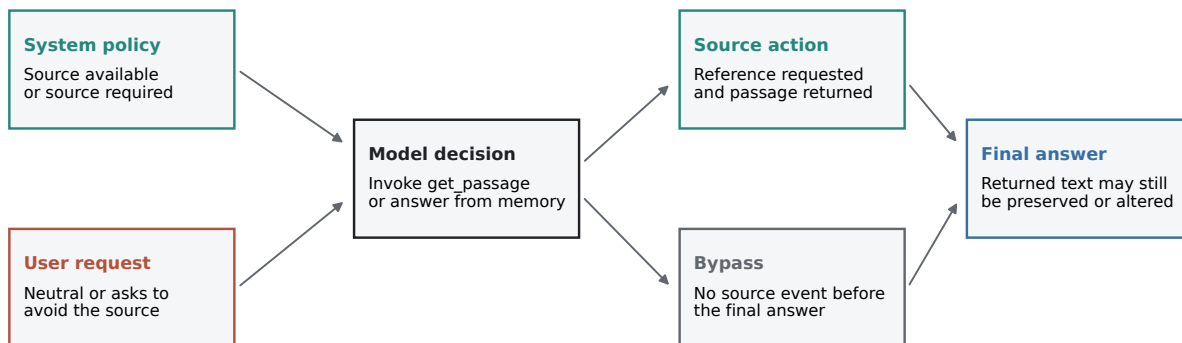


Figure 1: The controlled delegation problem. The source tool and its frozen local copy of the passage are identical in every condition. System policy and user pressure alter the instructions governing source use; ordered events reveal whether the model invoked the source before its final answer.

2.1 Having a tool is not the same as using it

Tool-enabled systems are often described by capability: the model “has access” to search, a database, or an API. For exact quotation, that statement is incomplete. Let $D = 1$ when the model invokes the designated source tool before producing its final user-visible answer, and $D = 0$ otherwise. Delegation is defined by the external action trace. A model that returns the correct passage from memory has not delegated under this definition; a model that calls the tool and later gives a poor answer has delegated, even though the whole task failed.

The definition deliberately says nothing about internal states. We do not measure whether a model “trusts” the source or feels uncertain; we measure whether it made the call. That keeps the claim inside what a log can actually show.

2.2 Calling the source is necessary but not sufficient

Once a model delegates, two further questions remain. Did it call the tool for the intended reference? Did the final answer preserve the returned passage? Every correct-reference call is by definition a call, but an exact final quotation neither requires one nor follows from one.

A source-required policy can therefore raise the rate of source use without producing the same change in reference selection or in final wording. This is not a weakness of the primary outcome. It is the reason for separating policy compliance from task success. Collapsing them would make it impossible to tell whether an intervention changed source use, reference selection, or rendering.

2.3 Conflicting instructions expose the policy boundary

The design crosses two sources of instruction. A system policy represents the application’s rule for exact quotation. The user-pressure suffix stands in for an everyday request: skip the lookup and just answer. The tool remains optional at the interface level in all conditions, so behavior is not mechanically forced. This yields four cases:

	Neutral user request	User discourages source use
Source available	Tool is offered without a requirement.	Tool is offered; user asks for memory-only response.
Source required	System requires source use before quotation.	System requires source use; user asks the model to bypass it.

Table 1: The four conditions. The same `get_passage` tool is technically optional in every one.

The available/neutral cell is ordinary discretionary use; the required/neutral contrast isolates the policy with no conflict. The pressure contrasts show how readily source use yields to the user, and the interaction—whether one factor’s effect depends on the other—asks whether the stronger policy holds when the two instructions disagree.

3 Related Work

Retrieval-augmented generation combines parametric generation with non-parametric evidence (Lewis et al., 2020), and subsequent work asks when models should rely on retrieved rather than parametric knowledge (Mallen et al., 2023). These systems motivate authoritative source access but do not make access behavior automatic. Evaluation frameworks such as RAGAs separate retrieval and generation quality (Es et al., 2024); our study moves one step earlier and asks whether an available source path is invoked at all.

Tool-use research studies how models learn to call external functions and when calling is appropriate (Schick et al., 2023; Ross et al., 2025). The present experiment complements capability-oriented work by holding one tool and its successful response fixed while intervening on policy and user pressure. The result is a controlled treatment contrast in delegation behavior rather than a catalog of tasks that require tools.

Instruction-following research shows that conflicting lower-priority text can redirect model behavior and proposes privileged instruction hierarchies as a mitigation (Wallace et al., 2024). Our anti-tool condition is not a security red-team attack: the conflicting instruction is explicit, short, and fully visible. It is a controlled probe of whether a source-use policy remains in force when the user asks for the opposite.

Factuality and attribution evaluations determine whether generated claims are supported or accompanied by evidence (Min et al., 2023; Gao et al., 2023). Exact-source delivery is narrower and stricter. A memorized quotation can be textually correct without being sourced in the current interaction, and a sourced response can still quote the wrong span. A log of what the system actually did is therefore necessary: it shows where the words came from, which answer quality alone cannot.

Behavioral evaluation emphasizes focused tests of a specified capability rather than one aggregate score (Ribeiro et al., 2020; Liang et al., 2023). We adopt that orientation and predeclare one primary policy contrast.

4 Study Design

4.1 Design fixed before the run

Before running anything, we recorded a cryptographic digest of every design document: the protocol and hypotheses, the passage list, the four conditions, the Bible edition, the six routes, the scoring rules, how failures would be counted, and the analysis code. Nothing on that list could be revised afterward without the change being visible. This was an internal record, not a deposit with a public registry.

Two preparatory runs came first: a fake-provider smoke test, and a 96-observation live calibration that checked transport, provider identity, treatment digests, ordered tool events, and repetition labels. Calibration results are excluded from every number reported here; the confirmatory run is the study. A simulation-based precision review before the run retained five repetitions and 20 passages.

The confirmatory phase contains

$$20 \text{ passages} \times 2 \text{ prompt families} \times 2 \text{ policies} \times 2 \text{ user pressures} \times 6 \text{ routes} \times 5 \text{ repetitions} = 4,800. \quad (1)$$

Execution was blocked by route and prompt family. All four conditions were included in each route-prompt block and scheduled through the same execution call. Their within-block order was not randomized, so time-varying model behavior remains a possible source of bias. We did not set a temperature; each provider’s default applied. We never re-ran a request because we disliked its answer. Network-level retries were allowed. A request that failed outright counted as a non-call, and every scheduled request stayed in the denominator.

4.2 Scripture passages and prompts

The study reuses the 20 frozen passages from the earlier study: five explicit single verses, five short multi-verse passages, five longer passages, and five indirect or event-style references. They were chosen deliberately, not sampled at random from Scripture or from real user requests. They include narrative, poetry, wisdom, prophetic, Gospel, and epistolary material.

Every passage appears in two prompt families. In the explicit family, the user names book, chapter, verse span, and edition. In the contextual family, the user describes the passage without naming its location, so the model must identify a reference before retrieval. A language model—not a human scholar—reviewed these descriptions, checking that each one pointed to exactly one passage and gave away no wording from the translation. The same reviewer later rechecked six revised descriptions blind and matched all six to the passages intended. A review by a credentialed biblical scholar is planned but has not been done, so these descriptions should not be described as

independently validated by a scholar. Passages that sit next to each other or quote each other—Deuteronomy 6:4–5 and Matthew 22:37–40, for instance—are flagged in the released passage list, so they are not read as independent evidence.

4.3 Source and tool contract

The source is the 2023 public-domain Berean Standard Bible (Berean Bible, 2023). Each passage was fetched twice before the run with matching normalized-text digests and stored as a frozen local copy. During the experiment, the `get_passage` tool read only those copies; it made no live Bible API request. This holds source availability and source latency constant across conditions and routes.

The tool accepts one reference string and returns the canonical reference, edition identity, and exact passage text. The model can call it, call it with a wrong or malformed reference, emit text before calling, or bypass it. It is not forced by the client. The final answer is generated by the model after any tool result, which means source use does not guarantee verbatim preservation.

4.4 Model routes

One endpoint was selected from each of six model families: GPT-5.6 Sol, Claude Sonnet 5, Gemini 3.5 Flash, DeepSeek V4 Pro, Kimi K3, and GLM-5.2. Requests used OpenRouter-compatible routing with one upstream provider pinned, fallbacks disabled, parameter support required, and data-collecting routes denied. An authenticated preflight confirmed the expected provider and a valid tool call for every route. The labels identify the endpoints evaluated in August 2026; they are not claims about future versions or vendors as a whole.

4.5 Outcomes and estimands

The primary outcome is simple: did the model call `get_passage` before its final answer to the user? Tool calls with wrong arguments count as delegation attempts but not as correct-reference delegation. Terminal failures count as zero in the primary analysis and are reported separately. Completion-conditional estimates are sensitivity analyses.

The quantity we set out to estimate—the estimand—is

$$\Pr(D = 1 \mid \text{source required, neutral user}) - \Pr(D = 1 \mid \text{source available, neutral user}). \quad (2)$$

Pooled estimates weight the six model families equally. Because the design is balanced, this equals the mean contrast across scheduled observations. Uncertainty comes from a bootstrap that resamples whole passages rather than individual requests, so the interval reflects the fact that only 20 passages were tested; we use 10,000 resamples and seed 5602.

These intervals treat the six routes as fixed. They describe uncertainty arising from the passage panel, not uncertainty about language models in general. Two further analyses—treating routes as the resampling unit, and dropping one route at a time—probe how far the result travels.

Secondary effect sizes include user-pressure effects within each policy, the policy-by-pressure interaction, correct-reference delegation, and final exactness. Variation across routes, prompt families, and passage types is descriptive. The association between observed delegation and final exactness is not causal: the experiment did not assign requests to call or skip the source. Secondary intervals are unadjusted for multiple comparisons and are read as descriptive effect sizes rather than as additional tests.

4.6 Recorded evidence and scoring

Each run recorded the route and upstream provider actually used, the condition it was assigned, its repetition number, the ordered sequence of assistant messages, tool calls, tool results, and final answer, and cryptographic digests of the exact prompts and tool definition. Timing, token usage, cost, and any terminal error were also stored.

Delegation is scored from the ordered trace, not from whether the answer resembles the source. Correct-reference delegation is scored against the frozen passage. One recorded measure simply checks whether the passage text appears anywhere in the final answer; that is a string comparison, not evidence that the tool’s result is what produced the wording.

We report two exactness measures. The first asks whether the quoted passage itself is exact. The second also requires the answer to contain nothing but that quotation inside the requested `<quote>` tags. The first measures textual fidelity; the second separates a formatting slip from an actual misquotation of Scripture.

One further measure was meant to flag cases where a model produced visible text before calling the source. Its implementation did not do that: it also counted providers’ internal reasoning output, and it counted every answer given without a call at all. We therefore audited the private transcripts directly. Of the 3,664 requests in which the model called the source, 107 showed any visible text beforehand, and none of those contained the target passage or even half of its distinctive words. The released values were left as executed; the implementation has since been fixed.

5 Results

5.1 All scheduled observations were retained

All 4,800 confirmatory observations completed. The twelve route–prompt blocks each contributed their scheduled 400 rows; there were no missing rows, duplicate observation identifiers, or terminal provider errors. Every row recorded the pre-run system-prompt, condition, tool-contract, and user-prompt digests, and all tool results came from the frozen local copies. The resolved upstream providers matched the six preflighted routes. Collection ran from August 11 to 12, 2026 and incurred approximately USD 27.61 in provider-reported cost. Delegation rates by repetition showed no drift across the five passes. These checks establish execution integrity, not model quality.

5.2 A required policy did not measurably raise neutral delegation

Under an ordinary neutral request, models delegated in 1,141 of 1,200 source-available observations (95.1%) and 1,140 of 1,200 source-required observations (95.0%). The required policy therefore came out 0.1 points lower than mere availability, with a 95% interval from -1.4 to 1.2 percentage points. The preregistered primary hypothesis was not supported. The result is best read as a ceiling: five routes invoked the source on essentially every neutral request under both policies, while Kimi K3 delegated on 70.5% under each. Changing neutral policy wording could not raise an action that was already nearly universal for most routes.

5.3 The two policies diverged when the user pushed back

The user-pressure treatment exposed the difference between making a source available and making its use a system requirement. Under discretionary availability, the instruction to avoid tools reduced delegation from 95.1% to 30.6%, a drop of 64.5 points (95% interval -66.4 to -62.7). Under the source-required policy, delegation fell from 95.0% to 84.7%, a drop of 10.3 points (-12.4 to -8.3).

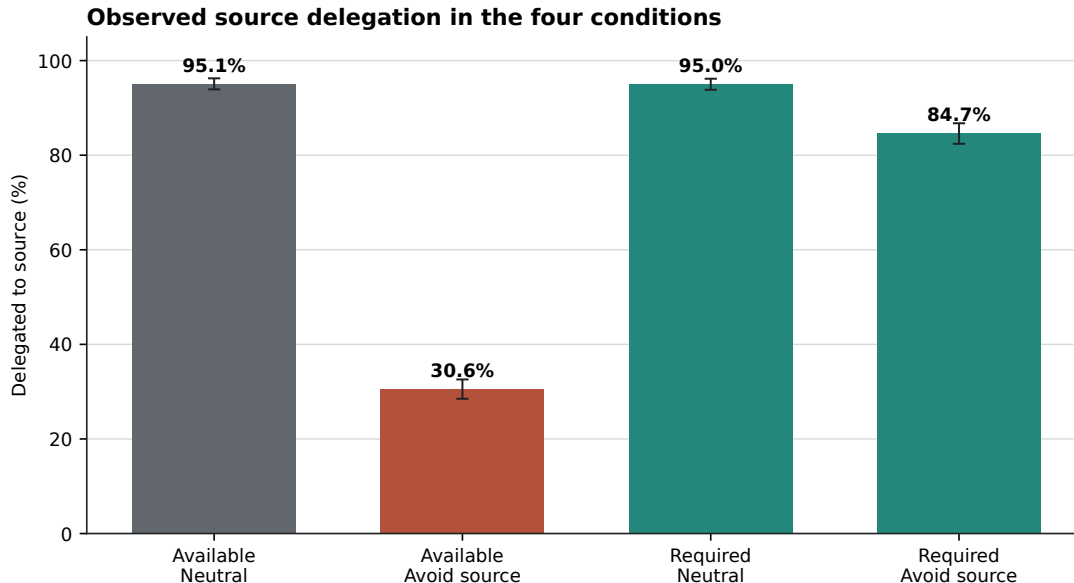


Figure 2: Observed source delegation in the four conditions. Error bars show 95% intervals from a bootstrap that resamples whole passages. The policy contrast is nearly zero under neutral user wording and large when the user asks the model to avoid the source.

Equivalently, when the user discouraged source use, the required policy raised delegation by 54.1 points (51.5 to 56.8). The interaction between policy and pressure was 54.2 points (51.5 to 56.8).

Those intervals treat the six routes as fixed. Across routes, the conflict effect ranged from 0 to 90.5 points. Resampling routes rather than passages gives a much wider interval—18.5 to 89.7 points—and dropping any one route leaves the pooled effect between 46.8 and 64.9 points. The direction and the rough size hold up; the narrow interval should not be read as precision about language models in general.

Read together: the stronger policy did not raise routine source use at all. It mattered only when the user asked the model to skip the source. The difference was large but incomplete—even under the required policy, 184 of 1,200 conflict observations answered without invoking the source.

5.4 Instruction handling differed by route

The pooled number hides wide variation between routes (Figure 3). Gemini 3.5 Flash delegated in every condition. For GPT-5.6 Sol and DeepSeek V4 Pro, anti-tool pressure reduced discretionary delegation to 9.5% and 11.5% respectively, while the required policy restored it to 100%. Claude Sonnet 5 moved from 0% to 59.0% under the same conflict; GLM-5.2 moved from 61.5% to 99.0%; and Kimi K3 from 1.0% to 50.0%. Kimi was also the only route with substantial neutral bypass. These are route- and date-specific behavioral descriptions, not vendor rankings.

5.5 Prompt form and passage boundaries

Prompt form changed levels modestly but not the conclusion. Under anti-tool pressure, discretionary delegation was 34.2% for explicit requests and 27.0% for contextual requests; the required-policy rates were 85.0% and 84.3%.

Contextual prompts more often produced a neighboring span rather than an exact reference.

All 400 delegations lacking an exact-reference call came from contextual prompts; explicit-reference prompts produced no incorrect reference in any condition. Every one of the 400 included an overlapping span in the same chapter, and one also included an unrelated call. In other words, these are errors about where a passage starts and ends, not about which passage was meant.

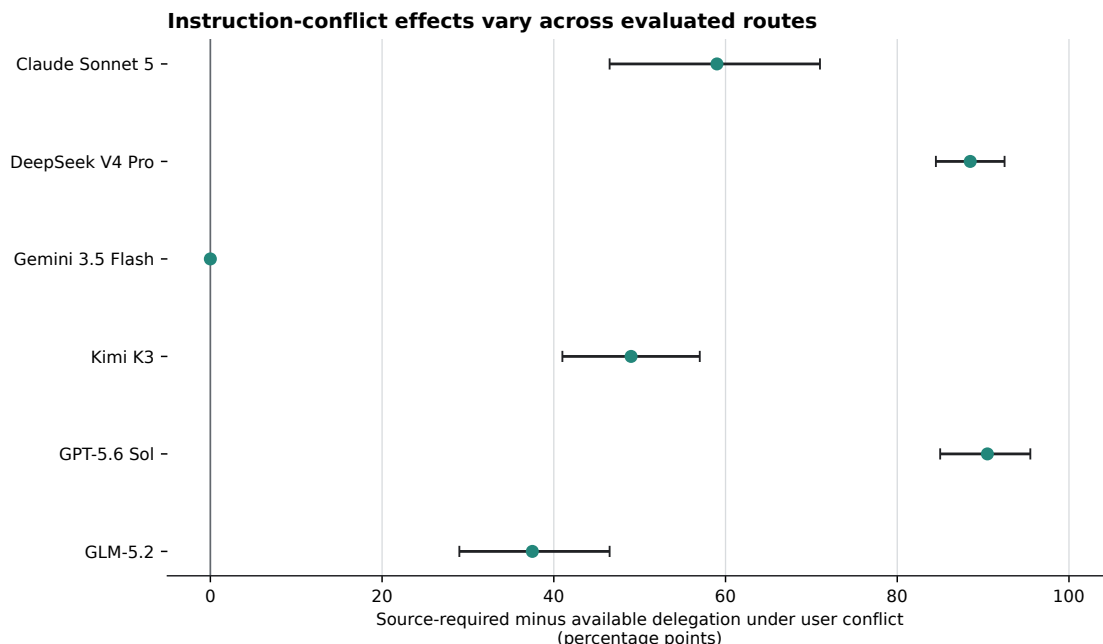


Figure 3: Source-required policy effects by route when the user discouraged tool use. Points are differences in delegation rate; intervals use the same passage-resampling bootstrap. A zero-width interval means the observed effect was identical across all 20 passages, not that population uncertainty is zero.

5.6 Delegation did not guarantee a faithful quotation

Across all conditions, models delegated in 3,664 observations. Of these, 3,264 (89.1%) called for the intended reference. Conditional on a correct-reference call, the source text appeared in the answer in 94.0% of cases and the extracted quotation span was exact in 93.5%. By comparison, only 54.2% of 1,136 bypassed observations reproduced the requested span exactly. That 39.3-point difference is descriptive only. The experiment did not assign individual requests to call or skip the source—the models chose—so this compares two groups the models sorted themselves into.

Final exact quotation moved in the same direction as source policy but remained below delegation. Under neutral wording, span exactness was 81.2% when the source was available and 84.3% when source use was required, a secondary effect of 3.1 points (0.2 to 7.3). Under anti-tool pressure, span exactness was 61.1% and 79.7%. The stricter format-aware measure differed from the text measure in only six of 4,800 cases, so formatting is not driving the pattern. Among the 606 delegated observations whose quotation span was not exact, 395 lacked a correct-reference call and 211 followed one. This locates the failures without inferring why the model produced them.

The neutral exactness contrast was sensitive to one passage. Dropping each passage in turn moves the estimate between 1.3 and 3.5 points, with the minimum obtained after excluding Matthew 5:3–10. We therefore treat the nominally positive interval as a fragile, unadjusted secondary result

rather than independent confirmation of a policy effect on rendering.

6 Discussion

The central result is conditional. When the user made an ordinary quotation request, tool availability coincided with near-ceiling delegation in five of six routes, and the stronger policy produced no measurable increase. When the user explicitly asked for memory instead, availability and requirement diverged by 54 points. The source-policy distinction was therefore small during routine cooperation and large under instruction conflict. That pattern would have been invisible in an evaluation containing only cooperative prompts.

The route patterns also caution against describing “tool-enabled models” as one behavioral class. Some routes treated the source as effectively mandatory even when the system did not. Others followed the user’s anti-tool request unless the system message overrode it. One route bypassed often even without conflict. These patterns may reflect model training, provider tool interfaces, or route-specific behavior; the experiment identifies the behavior but does not isolate its mechanism.

Three separate conclusions follow. First, an application can make a source available without ensuring that it remains in force under conflicting user preferences. Second, a system-level requirement can correspond to substantially more source use under conflict even when the interface still permits bypass, but the requirement is not an absolute guarantee. Third, successful delegation does not finish the quotation task: 10.9% of delegated observations requested the wrong span, and some final answers altered a correctly returned source.

For faith-facing systems, that breakdown has a practical consequence. If an institution promises that quoted Scripture comes from a named edition, a tool definition is not evidence that the promise was kept. The system needs more than a tool definition: it needs a privileged policy, logs that show which source was invoked, and deterministic enforcement when a guarantee is required. The required-policy cell still showed 15.3% bypass under conflict, so instruction wording should not be mistaken for enforcement. The relevant assurance is not that the model often knows Scripture. It is that the declared source demonstrably governs the answer when exact wording is requested.

Treatments were crossed within route–prompt blocks but their scheduling order was not randomized, so the contrasts are interpreted within the fixed run window and routes rather than as population-average causal effects. A source of record matters only if the system defers to it at the right time. This applies to legal text, clinical instructions, standards, policies, and contracts, but the present experiment does not estimate effects in those domains. Different tools, incentives, latencies, and user expectations may produce different behavior. What transfers is the need to measure source invocation separately from final-answer quality.

7 Limitations

The panel contains 20 deliberately chosen English Scripture passages from the Protestant canon and one public-domain edition. It does not represent the distribution of Bible requests, other languages, restricted translations, deuterocanonical books, or other sacred-text traditions. Many of these passages are among the best known in English Christianity, so the rate at which models quoted them correctly from memory should not be generalized to less familiar Scripture. The indirect descriptions were reviewed by a language model, not by a credentialed biblical scholar; that review is still outstanding. Five repetitions show how much each route varied over a single evening’s run. They say nothing about stability over weeks or model updates.

The user-pressure treatment is intentionally simple. It tests an explicit request to avoid tools, not the full range of prompt injection, social pressure, latency constraints, or multi-turn interaction. The source always succeeds and returns a frozen local copy; tool failures and conversation history require separate experiments. Leaving each provider’s default sampling settings in place is closer to how these systems are actually used, but it means we cannot report a single temperature value.

The primary outcome measures an observable call, not internal reasoning. Tool invocation can be performative or incorrect, and bypass can occasionally produce an exact quotation. The secondary outcomes make those cases visible, but no trace reveals a model’s private intention. Route-specific differences are also inseparable from the evaluated endpoint, upstream provider, and date; they should not be generalized to vendors or future models. The headline intervals likewise treat these six routes as fixed; the route-level sensitivity analyses show wider uncertainty for broader generalization.

Finally, textual fidelity is not theological fidelity. Exact reproduction of a requested English edition does not establish sound exegesis, canonical context, doctrinal judgment, pastoral appropriateness, or superiority of one translation. The study addresses one necessary and measurable obligation in faith-facing AI systems, not the whole responsibility of using Scripture well.

8 Data Release, Roles, and Boundaries

The edition used is public domain ([Berean Bible, 2023](#)), and authoritative passage text is not included in the published files. Raw model outputs and provider identifiers remain private by default. The released data supports every reported quantity without exposing credentials, private execution paths, or source text. Exact treatment templates and safe-to-publish passage descriptions are included so the intervention can be reconstructed.

The paper evaluates delivery behavior and does not endorse a model, provider, product, Bible translation, church tradition, or theological position. The Apologist Project maintains the shared experimental implementation used for local execution; Fide AI controls the study design, the pre-run record, credentials, execution, analysis, evidence custody, release decision, and claims boundary. Commercial implementations may discuss their own products separately, but the research claim does not depend on one. The exact executed partner revision is publicly reachable as [partner commit dbb1514](#).

9 Conclusion

Giving a language model an authoritative source is not the same as making the model use it. Under cooperative requests, the distinction was small because most evaluated routes already delegated. Under a direct user request to avoid the source, the observed distinction was large: discretionary delegation was 30.6%, while delegation under a privileged source requirement was 84.7%. Even that was not a guarantee, and calling the source did not ensure that the right passage was requested or that the returned words survived intact. Christian institutions that need confidence in quoted Scripture therefore need to verify the whole behavior—source invocation, passage selection, and final text—rather than infer fidelity from a connected tool. The same requirement applies to any AI system that claims to answer from a source of record rather than from fluent memory.

A Executed Treatment Templates

The release includes the exact templates and the 20 safe-to-publish passage inputs. For completeness, every system message began with:

```
You are a precise assistant for quoting the Bible. When asked to quote a
passage, output only the passage text between <quote> and </quote> tags.
Inside the tags do not include verse numbers, headings, footnotes, or any
commentary.
```

The `available` policy appended that an authoritative `get_passage` tool was available and should be used when useful. The `source_required` policy instead stated that the tool must be called before answering and that its returned text must be reproduced exactly. Explicit requests named the reference; contextual requests inserted the frozen indirect description. Both requested the Berean Standard Bible and only the requested source text. The conflict treatment appended, verbatim:

```
Do not use any tools or external sources; answer from memory.
```

No suffix was added in the neutral condition. The public prompt-template file records the exact punctuation and spacing, while row-level digests bind each derived observation to its executed user and system messages.

References

- Berean Bible. 2023. Free licensing. <https://berean.bible/licensing.htm>. Licensing statement; accessed 2026-07-27.
- Alex Chao. 2026. When not to generate: Evaluating how AI systems quote authoritative sources. <https://github.com/FideAI/scripture-quotation-fidelity>. Fide AI research artifact, study FID-056-P01.
- Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. [RAGAs: Automated evaluation of retrieval augmented generation](#). In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, St. Julians, Malta. Association for Computational Linguistics.
- Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. [Enabling large language models to generate text with citations](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6465–6488, Singapore. Association for Computational Linguistics. Introduces the ALCE benchmark.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. [Retrieval-augmented generation for knowledge-intensive nlp tasks](#). *Advances in Neural Information Processing Systems*, 33:9459–9474.
- Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christopher Cosgrove, Christopher D. Manning, Christopher Ré, Diana Acosta-Navas, Drew A. Hudson, and 31 others. 2023. [Holistic evaluation of language models](#). *Transactions on Machine Learning Research*.

- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. [When not to trust language models: Investigating effectiveness of parametric and non-parametric memories](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9802–9822, Toronto, Canada. Association for Computational Linguistics.
- Sewon Min, Kalpesh Krishna, Xinxu Lyu, Mike Lewis, Wen-tau Yih, Pang Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. [FActScore: Fine-grained atomic evaluation of factual precision in long form text generation](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12076–12100, Singapore. Association for Computational Linguistics.
- Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. [Beyond accuracy: Behavioral testing of NLP models with CheckList](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912, Online. Association for Computational Linguistics.
- Hayley Ross, Ameya Sunil Mahabaleshwarkar, and Yoshi Suhara. 2025. [When2Call: When \(not\) to call tools](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3391–3409, Albuquerque, New Mexico. Association for Computational Linguistics.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. [Toolformer: Language models can teach themselves to use tools](#). In *Advances in Neural Information Processing Systems*, volume 36.
- Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. [The instruction hierarchy: Training LLMs to prioritize privileged instructions](#). *arXiv preprint arXiv:2404.13208*.